Constructor University Bremen

Natural Science Laboratory Communications

Fall Semester 2025

Lab Experiment 4

Author of the report Mr. Nikoloz Burduli

Date of execution : November 17, 2025

1. What are the two functions of a PPL in a communications system?

A PLL in a communications system mainly serves two purposes. First, it helps with carrier recovery, where the local oscillator is adjusted so it follows the phase and frequency of the incoming signal. Second, it is used for symbol timing recovery, making sure the receiver takes its samples at the right moments at the output of the matched filter.

2. What are the key components (operations) used to implement a PPL?

A PLL is built from a few essential parts that work together to track the phase of an incoming signal. The first part is the phase detector, which compares the reference signal with the signal coming from the oscillator and produces a value that reflects how far out of phase they are. This output contains both a useful low-frequency term and an unwanted high-frequency term from the sinusoid multiplication. The next part is a low pass filter, which removes the high-frequency component and keeps only the slower term that represents the actual phase error. The filtered signal then goes into the voltage controlled oscillator, which changes its frequency depending on the error. If the oscillator is falling behind it speeds up, and if it is ahead it slows down. When the two signals become aligned, the correction signal approaches zero, showing that the loop has locked.

3. When the second order frequency response of the PLL is written out, that formula basically shows how the phase at the input of the loop is connected to the phase at the output. In other words, it describes how the PLL reacts to changes in the reference phase and how quickly the loop can follow those changes. It captures the dynamic behaviour of the loop, like how fast it settles and how much it might overshoot before locking.

4. What does the corner frequency response of the PPL, this expression relates the input and output phase of the references.

The corner frequency of the PLL loop filter basically sets how quickly the loop can react to any phase shifts. If this frequency is higher, the PLL follows changes more fast, but it might get a bit noisy. If it is lower, the loop reacts slower but becomes more stable and less sensitive to jitter. So it is like a trade-off between speed of tracking and smoothness of the output.

5. How should the loop filter corner frequency compare to the input reference frequency?

The loop filter corner frequency is usually set quite a bit lower than the reference frequency, something like one tenth of it or even less. This helps the PLL stay stable and not chase every tiny variation in the input.

6. What does it mean for a PLL to track?

When a PLL is tracking, it basically means the loop has settled so the phase difference between the VCO and the incoming signal stays constant in time, ideally really close to zero. In this state the VCO is running at the right speed and phase.

7. What does it mean for the PLL loop to be overdamped or underdamped?

If the PLL loop is overdamped, it reacts slowly to any phase changes and needs longer to settle but it avoids extra oscillation. If it's underdamped, the loop reacts quicker but tends to overshoot and the phase may kind of ring or oscillate before it settles.

8. Why is an accumulator useful for a PLL implementation?

An accumulator is useful because it keeps the running phase value from one step to the next. The lookup table only has one cycle of the sine wave, so the accumulator tells the system exactly where inside that cycle it is at any moment.

9. Why do we need to sample sin() in our lookup table more finely than at the normal sample rate of the system?

We sample the sine function in the table more densely than the system's normal sample rate because the PLL works with fractional phase values. If the table is too coarse, the output gets noisy or jumpy. More points in the table gives smoother and more accurate phase tracking.

10. How do we keep our accumulator in the range [0,1]?

To keep the accumulator inside the range from 0 to 1, we simply wrap it around. The line s.accum = s.accum - floor(s.accum) gets rid of the integer part so what stays is only the fractional value, which always lies inside that range.

11. What is the point of storing and restoring the state of the PLL for each block?

Storing and restoring the PLL state for each block makes the algorithm continue smoothly. It preserves whatever phase and error information was built up in the previous block so the next block doesn't start from zero and break the tracking.

12. How do we handle signals with arbitrary amplitude?

With signals that come in with random amplitudes, we need to normalize them. In the Matlab code the amplitude is estimated and then the sample is scaled by that estimate. This stops changes in amplitude from messing with the phase estimation inside the PLL.

13. A paper design of your PLL, showing the important parameters that are needed for implementation

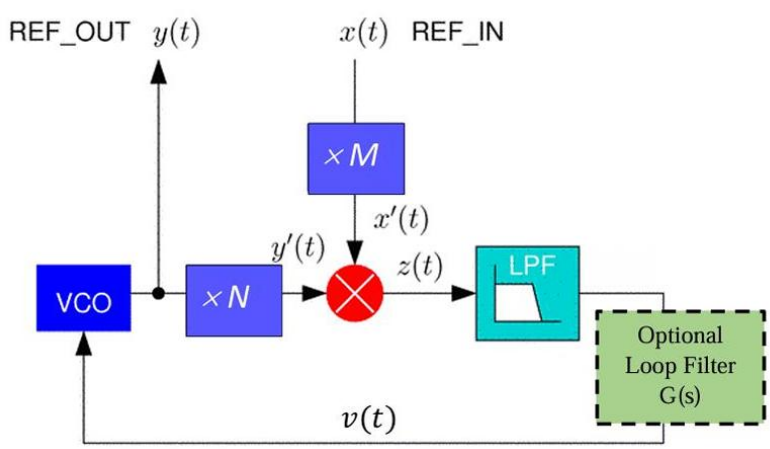

Figure 1. Paper Design of PPL

As it can be seen on the picture above, the PLL is made up from a few key parts, like the low pass filter and the VCO. The reference signal has to go through these stages so the loop can adjust itself and finally reach correct tracking of the phase.

Parameter	Description	Value
f	Reference Frequency	0.1 Hz
D	Damping Factor	1
K	Loop Gain	1
W_0	Loop Corner Frequency	2Pi/100 rad/s
T	Sampling Period	10 s
S_r	Sampling Rate	8000 Hz
n	Number of Samples	1024

Table 1. Parameters for the Implementation

14. A printout of your final MATLAB implementation of the PLL

```
% PLL1.m

% Global parameters
f = 0.1;       % Nominal eference frequency
D = 1.0;       % Damping factor
k = 1.0;       % Loop gain
omega = 2 * pi / 100; % Loop corner frequency
T = 1 / f;     % Sampling period
s_r = 8000;    % Sampling rate
% Time constants
tau1 = k / (omega * omega);
tau2 = ((2 * D) / omega) - (1 / k);
% The coefficients for the filter
a1 = -((T - 2 * tau1) / (T + 2 * tau1));
b0 = (T + 2 * tau2) / (T + 2 * tau1);
b1 = (T - 2 * tau2) / (T + 2 * tau1);
x = sin(2 * pi * f * (0:1024)); % reference input signal
y = zeros(1, 1025); % Allocates space for the output and fills it with zeroes
y(1) = 0; % Initial condition
% loops through each sampling coefficient and applies the function to it
for n = 2:1024
    y(n) = a1 * y(n-1) + b0 * x(n) + b1 * x(n-1);
end

% Plot input and output signals
figure;
plot(0:1024, x, 'b');
hold on;
plot(0:1024, y, 'r');
xlabel('Sample Coefficient');
ylabel('Amplitude');
legend('Reference (Input) Signal', 'PLL Output');
grid on;
hold off;
```

```matlab
% PLL_init1.m
function [state] = PLL_init1(f, D, k, omega, T, table_size) % initializes a

% Save parameters: nominal reference frequency, damping factor,
% loop gain, loop corner frequency, sampling period, sampling rate
state.f = f;
state.D = D;
state.k = k;
state.omega = omega;
state.T = T;

% Generate the lookup table
for ii = 0:table_size-1
    state.sin_table(ii+1) = sin(2 * pi * ii / table_size);
end

% Filter parameters for PLL implementation
state.t1 = state.k / (state.omega^2);
state.t2 = (2 * state.D / state.omega) - (1 / state.k);
state.a1 = -(state.T - 2 * state.t1) / (state.T + 2 * state.t1);
state.b0 = (state.T + 2 * state.t2) / (state.T + 2 * state.t1);
state.b1 = (state.T - 2 * state.t2) / (state.T + 2 * state.t1);

% Tracking variables
state.accum = 0; % the accumulator keeps track of the current phase
state.amp = 1; % amplitude value
state.err = 0; % stores the last obtained error for further adjustments
state.filt = 0; % save last filter uotput
state.out_p = 0; % save last PLL output
end
```

```matlab
%PLL_def1.m

function [state_out, y] = PLL_def1(state_in, x, N) % Processes N samples of the input signal using the PLL

% Initialization of output buffer filled with zeroes as well as variables for PLL tracking
s = state_in; % variable to store the input PLL state
y = zeros(size(x));
amp = 0;

for i = 1:N
    amp = amp + abs(x(i)); % Calculate amplitude
    scale = x(i) / s.amp;
    e = scale * s.out_p; % Calculate phase error

    % Apply loop filter to the filter coefficients and estimated error and filter output for better tracking
    v = s.a1 * s.filt + s.b0 * e + s.b1 * s.err;
    % Increment phase accumulator
    s.accum = s.accum + s.f - (s.k / (2 * pi)) * v;
    s.accum = s.accum - floor(s.accum); % Keep accumulator in range [0,1]

    y(i) = s.sin_table(floor(1024 * s.accum) + 1);

    % Store last values
    s.out_p = y(i);
    s.err = e;
    s.filt = v;
end
s.amp = amp / N / (2 / pi);
state_out = s; % variable to store the output PLL state
end
```

```
% PLL_test1.m

Ns = 100; % Number of samples
Nb = 10; % Number of blocks
[p] = PLL_init1(0.1, 1, 1, 2 * pi / 100, 10, 1024);
load('ref_stepf.mat');
load('ref_800hz.mat');
in = reshape(ref_in, Ns, Nb); % Input signal reshaped into Ns rows and Nb columns
out = zeros(Ns, Nb); % Allocates the space for the output and fills it with zeroes

% applies the PLL function to each block using the loop and stores the result
for n = 1:Nb
    [p, out(:, n)] = PLL_def1(p, in(:, n), Ns);
end
y = reshape(out, Ns * Nb, 1); % reshapes the output in the single vector
x = ref_in; % stores the reference input signal

% plots the result
figure;
plot(1:length(x), x, 'b');
hold on;
plot(1:length(y), y, 'm');
xlabel('Sample Index');
ylabel('Amplitude');
legend('Reference Signal', 'PLL Output');
grid on;
hold off;
```

15. A plot showing the simulated performance of the final PLL, demonstrating that the PLL can adapt to abrupt changes in frequency/phase

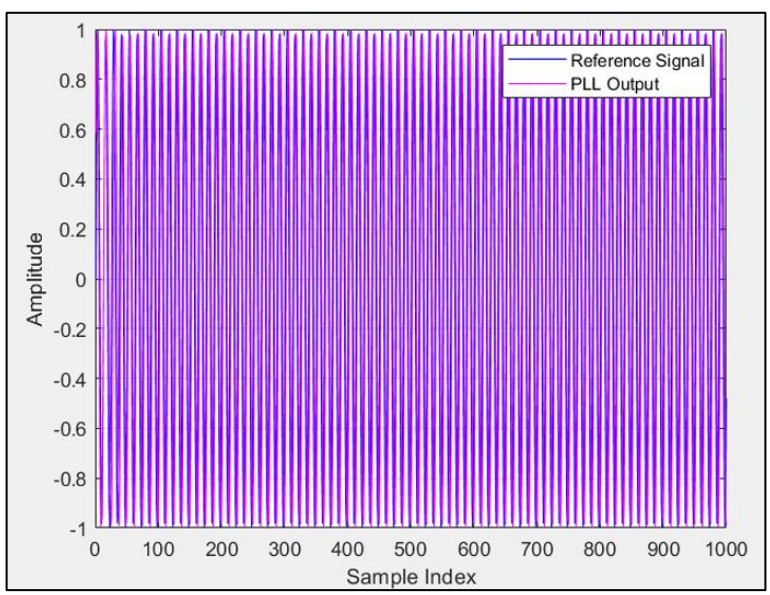

Figure 2. Final PPL - Simulated

From the plot above we can see that the PLL we implemented is behaving the way it should. It manages to follow the phase and also the frequency of the reference signal quite well, so the tracking is basically working as expected.

References:

Lab Manual Communication Basics "Delay and FIR (Matlab Part)", Ph. D. Fangning Hu.